

Fall 2026 Exploratory Data Analysis with R

Lecture 2: Data Manipulation and the Tidyverse

Brian Robertson
Federal Reserve Bank of Atlanta
2026-09-04

01

Recap & Agenda

01 Recap & Agenda

- 02 The Pipe & Core Verbs
- 03 Creating New Variables
- 04 Summarizing by Group
- 05 Reshaping Data
- 06 Recap

Quick Recap: Last Week

- Got Posit Cloud set up and wrote our first lines of R
- Imported a dataset with `read_csv()`
- Took a first look at `unemp_df`: the U.S. unemployment rate, 1948–2022
- Got a first taste of `summary()` and `filter()`

Today: turn that first taste into real fluency.

Goals for Today

- Meet the pipe, `%>%`
- Learn the five core **dplyr** verbs: `select()`, `filter()`, `arrange()`, `mutate()`, `summarize()`
- Pair `group_by()` with `summarize()` to answer questions *by group*
- Reshape data from wide to long with `pivot_longer()`
- Apply all of it to `unemp_df` — and to the population data from Homework 1

02

The Pipe & Core Verbs

- 01 Recap & Agenda
- 02 The Pipe & Core Verbs**
- 03 Creating New Variables
- 04 Summarizing by Group
- 05 Reshaping Data
- 06 Recap

Reading Code Left to Right

Nested function calls get hard to read fast:

```
arrange(filter(unemp_df, Year > 2000), desc(Unrate))
```

```
# A tibble: 22 × 2
  Year Unrate
  <dbl> <dbl>
1  2010   9.61
2  2009   9.28
3  2011   8.93
4  2020   8.09
5  2012   8.07
6  2013   7.36
7  2014   6.16
8  2003   5.99
```

```
9  2008  5.8
10 2002  5.78
# i 12 more rows
```

To read this you have to work from the inside out. The **pipe operator**, `%>%`, lets us write the same thing as a sequence of steps, read top to bottom:

```
unemp_df %>%
  filter(Year > 2000) %>%
  arrange(desc(Unrate))
```

```
# A tibble: 22 × 2
   Year Unrate
  <dbl> <dbl>
1  2010   9.61
2  2009   9.28
3  2011   8.93
```

```
4 2020 8.09
5 2012 8.07
6 2013 7.36
7 2014 6.16
8 2003 5.99
9 2008 5.8
10 2002 5.78
# i 12 more rows
```


How to Read `%>%`

- Read it as “**and then**”
- `x %>% f()` is exactly the same as `f(x)`
- `x %>% f() %>% g()` is exactly the same as `g(f(x))`

Every function in the **tidyverse** is built to work this way: take data in as the first argument, hand data back out, ready for the next step.

Choosing Columns: `select()`

Sometimes a dataset has more columns than we need. `select()` keeps only the ones we ask for.

```
unemp_df %>%  
  select(Year, Unrate) %>%  
  head(4)
```

```
# A tibble: 4 × 2  
  Year Unrate  
  <dbl> <dbl>  
1  1948   3.75  
2  1949   6.05  
3  1950   5.21  
4  1951   3.28
```

`unemp_df` only has two columns already, so this one won't look like much yet — but wait until we bring in a dataset with more of them.

`select()` Tricks

- `select(df, -Year)` — keep everything *except* `Year`
- `select(df, starts_with("Un"))` — keep columns whose names start with `"Un"`
- Column order in, column order out: `select()` also **reorders**

Exercise 1: Filtering, Properly

Last week we used `filter()` once. Let's push it further.

1. Filter `unemp_df` to years **after** 2007 (the leadup to the financial crisis)
2. Filter to years where `Unrate` was **above 6**
3. Combine both conditions into a single `filter()` call

Combining Conditions

```
unemp_df %>%  
  filter(Year > 2007 & Unrate > 6)
```

```
# A tibble: 7 × 2  
  Year Unrate  
  <dbl> <dbl>  
1  2009   9.28  
2  2010   9.61  
3  2011   8.93  
4  2012   8.07  
5  2013   7.36  
6  2014   6.16  
7  2020   8.09
```

- `&` means **and** — both conditions must hold

- `|` means **or** — either condition can hold
- `%in%` checks membership: `Year %in% c(2008, 2009, 2020)`

Putting Things in Order: `arrange()`

```
unemp_df %>%  
  arrange(Unrate) %>%  
  head(5)
```

```
# A tibble: 5 × 2  
  Year Unrate  
  <dbl> <dbl>  
1  1953   2.92  
2  1952   3.02  
3  1951   3.28  
4  1969   3.49  
5  1968   3.56
```

That's the five **lowest**-unemployment years on record in this dataset. To flip the order:


```
unemp_df %>%  
  arrange(desc(Unrate)) %>%  
  head(5)
```

```
# A tibble: 5 × 2  
  Year Unrate  
  <dbl> <dbl>  
1  1982   9.71  
2  2010   9.61  
3  1983   9.6  
4  2009   9.28  
5  2011   8.93
```

arrange() + filter() Together

The real power shows up once you start chaining verbs:

```
unemp_df %>%  
  filter(Year >= 2000) %>%  
  arrange(desc(Unrate))
```

```
# A tibble: 23 × 2  
  Year Unrate  
  <dbl> <dbl>  
1  2010   9.61  
2  2009   9.28  
3  2011   8.93  
4  2020   8.09  
5  2012   8.07  
6  2013   7.36
```

```
7  2014    6.16
8  2003    5.99
9  2008    5.8
10 2002    5.78
# i 13 more rows
```

This reads as: *start with `unemp_df`, keep years from 2000 on, then sort by unemployment, highest first.*

Breakout Session 1

Task: Using `unemp_df` and the pipe, answer:

1. What are the three years with the **lowest** unemployment since 1990?
2. Was unemployment ever above 9% before the year 2000? Which years?
3. Sort all years from 1980–1989 by unemployment, worst to best.

Hint: every one of these is a `filter()` and/or `arrange()`, chained with `%>%`.

03

Creating New Variables

- 01 Recap & Agenda
- 02 The Pipe & Core Verbs
- 03 Creating New Variables**
- 04 Summarizing by Group
- 05 Reshaping Data
- 06 Recap

Creating New Columns: `mutate()`

Every verb so far has worked with columns that already exist. `mutate()` **creates** new ones.

```
unemp_df %>%  
  mutate(decade = (Year %/% 10) * 10) %>%  
  head(4)
```

```
# A tibble: 4 × 3  
  Year Unrate decade  
  <dbl> <dbl>   <dbl>  
1  1948   3.75   1940  
2  1949   6.05   1940  
3  1950   5.21   1950  
4  1951   3.28   1950
```

`%%` is integer division — `1987 %% 10 = 198`, times 10 gives `1980`. This is a common trick for bucketing years into decades.

mutate() with lag()

A very common use case: comparing a row to the row *before* it.

```
unemp_df %>%  
  mutate(change = Unrate - lag(Unrate)) %>%  
  filter(Year >= 2007 & Year <= 2010)
```

```
# A tibble: 4 × 3  
  Year Unrate  change  
  <dbl> <dbl>   <dbl>  
1  2007    4.62 0.00833  
2  2008    5.8  1.18  
3  2009    9.28 3.48  
4  2010    9.61 0.325
```


`lag(Unrate)` shifts the whole column down by one row, so `change` is the year-over-year move in the unemployment rate. Watch what happens around 2009.

Take Ten

Back at the top of the hour.

(Refill your coffee. Stretch. Do not open Canvas.)

04

Summarizing by Group

- 01 Recap & Agenda
- 02 The Pipe & Core Verbs
- 03 Creating New Variables
- 04 Summarizing by Group**
- 05 Reshaping Data
- 06 Recap

group_by() + summarize()

The single most useful pattern in the tidyverse: split your data into groups, then compute something for each group.

```
unemp_df %>%  
  mutate(decade = (Year %/% 10) * 10) %>%  
  group_by(decade) %>%  
  summarize(avg_unrate = mean(Unrate),  
            max_unrate = max(Unrate))
```

```
# A tibble: 9 × 3  
  decade avg_unrate max_unrate  
  <dbl>    <dbl>    <dbl>  
1  1940      4.9      6.05  
2  1950      4.51     6.84  
3  1960      4.78     6.69
```

4	1970	6.22	8.48
5	1980	7.27	9.71
6	1990	5.76	7.49
7	2000	5.54	9.28
8	2010	6.22	9.61
9	2020	5.71	8.09

`group_by()` doesn't change what the data looks like — it changes what happens **next**. `summarize()` then collapses each group down to one row.

Exercise 2: By Decade

Using the `decade` column from `mutate()`:

1. Find the decade with the **highest average** unemployment
2. Find the single **worst year** within that decade
3. Do it all in one piped chain — no saving intermediate objects

Breakout Session 2

Work with a partner.

Task:

1. Compute average unemployment for every decade from 1950 through 2019
2. Sort the result from highest to lowest average
3. Which decade would you *least* want to be graduating into? Why?

Stretch goal: instead of `mean()`, try `sd()` — which decade had the most *volatile* unemployment, not just the highest?

05

Reshaping Data

- 01 Recap & Agenda
- 02 The Pipe & Core Verbs
- 03 Creating New Variables
- 04 Summarizing by Group
- 05 Reshaping Data**
- 06 Recap

Wide vs. Long Data

Remember `decade_pop.csv` from Homework 1? Let's bring it back.

```
pop_df
```

```
# A tibble: 24 × 4
  Year    Black    White Source
<dbl>  <dbl>    <dbl> <chr>
1  1790   757208   3172006 Census https://www.census.gov/content/dam/Census/libr...
2  1800  1002037   4306446 Census https://www.census.gov/content/dam/Census/libr...
3  1810  1377808   5862073 Census https://www.census.gov/content/dam/Census/libr...
4  1820  1771656   7866797 Census https://www.census.gov/content/dam/Census/libr...
5  1830  2328642  10532060 Census https://www.census.gov/content/dam/Census/libr...
6  1840  2873648  14189705 Census https://www.census.gov/content/dam/Census/libr...
7  1850  3638808  19553068 Census https://www.census.gov/content/dam/Census/libr...
8  1860  4441830  26922537 Census https://www.census.gov/content/dam/Census/libr...
```

```
9  1870 4880009 33589377 Census https://www.census.gov/content/dam/Census/libr...
10 1880 6580793 43402970 Census https://www.census.gov/content/dam/Census/libr...
# i 14 more rows
```

This is **wide** data: each *row* is a decade, and *Black* and *White* are separate *columns*. It's easy to read, but awkward for the tidyverse — `ggplot()` and `group_by()` both want one **variable per column**, and here, “population” is really one variable split across two columns.

What We Want Instead

Long data would look like this:

Year	race	population
1790	Black	757208
1790	White	3172006
1800	Black	1002037
...	...	...

One row per Year–race combination. This is what `pivot_longer()` is for.

`pivot_longer()`

```
pop_long <- pop_df %>%  
  pivot_longer(cols = c(Black, White),  
               names_to = "race",  
               values_to = "population")  
  
pop_long %>% head(6)
```

```
# A tibble: 6 × 4  
  Year Source                                race population  
  <dbl> <chr>                                <chr>         <dbl>  
1  1790 Census https://www.census.gov/content/dam/Census/libra... Black          757208  
2  1790 Census https://www.census.gov/content/dam/Census/libra... White         3172006  
3  1800 Census https://www.census.gov/content/dam/Census/libra... Black          1002037  
4  1800 Census https://www.census.gov/content/dam/Census/libra... White         4306446
```

```
5 1810 Census https://www.census.gov/content/dam/Census/libra... Black 1377808
6 1810 Census https://www.census.gov/content/dam/Census/libra... White 5862073
```

- `cols` — which columns to fold together
- `names_to` — the new column that holds the old column *names* ("Black", "White")
- `values_to` — the new column that holds the values that used to live in those columns

Now `group_by()` Actually Works

```
pop_long %>%  
  group_by(race) %>%  
  summarize(pop_1790 = min(population),  
            pop_2020 = max(population),  
            growth = max(population) - min(population))
```

```
# A tibble: 2 × 4  
  race  pop_1790  pop_2020  growth  
  <chr>    <dbl>    <dbl>    <dbl>  
1 Black   757208  41100000  40342792  
2 White  3172006 223553265 220381259
```

That's the whole point of reshaping: once “race” is a column instead of two separate columns, `group_by(race)` is one line of code instead of a headache.

Breakout Session 3

Using `pop_long` from the last section:

1. Compute each group's **share** of the total population in 1900, 1950, and 2000 (`filter()` + `mutate()`)
2. Which decade had the fastest growth for each group? (Hint: you'll want `arrange()` inside each group — look up `group_by() %>% arrange()`)
3. In your own words: why was reshaping to long format a **necessary** step before you could answer question 2?

06

Recap

- 01 Recap & Agenda
- 02 The Pipe & Core Verbs
- 03 Creating New Variables
- 04 Summarizing by Group
- 05 Reshaping Data
- 06 Recap**

Recap

Today we added five verbs to the toolkit:

- `select()` — choose columns
- `filter()` — choose rows
- `arrange()` — reorder rows
- `mutate()` — create columns
- `group_by()` + `summarize()` — compute *by group*

And one reshaping tool: `pivot_longer()`, for turning wide data into tidy, analysis-ready long data.

Chained together with `%>%`, these five verbs cover the overwhelming majority of day-to-day data wrangling — in this class, and afterward.

Links & Further Reading

- [R for Data Science, Ch. 5: Data Transformation](#)
- [RStudio dplyr cheat sheet](#)
- [tidyr pivoting vignette](#)

Coming up: more summary statistics, and starting the project.